ML Engineer Assessment Task

Design and Prototype a Validated Dynamic Questioning System with Retrieval-Grounded Triggering and Parallel State Fusion (Vehicle Insurance)

Objective

Build a vehicle insurance claim intake + follow-up questioning engine that conducts a structured interview from user-provided information and produces:

- a validated sequence of questions (dynamic, turn-by-turn),
- a structured claim summary JSON, and
- an audit log that proves why each question was asked and why it was considered valid.

This is explicitly a vehicle insurance company environment with vehicle insurance claim inputs and supported followup questions.

Key Rule: Candidate must generate everything

No domain question bank, schemas, or metadata templates are provided as starter packs. The prototype must run using only the artifacts created by you.

What the final system does

At every turn, it takes a claimant/incident state like:

```
{
  "category": "collision",
  "loss_datetime": "Feb 3rd, 2026",
  "loss_location": {"city": "Pune", "road_type": "urban"},
  "vehicle": {"make": "Maruti", "model": "Baleno", "year": 2021},
  "drivable": true,
  "third_party_involved": true,
  "already_extracted_categories":"collision",
  "answered_question_ids": ["Q0008"]
```

```
}
```

and returns the best next question from a corpus of ~1200 pre-built questions:

```
{
  "question_id": "Q0345",
  "question_text": "Was the other vehicle's registration number captured
(photo/notes)?",
  "question_field":"third_party_details_registration",
  "trigger": "collision",
  "score_breakdown": {"priority": 0.6, "relevance": 0.8, "gap_fill": 0.7}
}
```

- The system must be input-driven and state-aware at every turn.
- If the user mentions a fact at any time, it must be extracted into state and suppress future questions targeting that field.
- Questions must shift when the incident type changes (e.g., user says “fire”), and **must not follow a fixed yes/no tree.**

Core concept definitions

Triggers vs Targets

Each question must include metadata such as (for reference):

- triggers = conditions that must be true to ask the question
- priority = integer 1 (highest) … 5 (lowest)
- already_extracted_categories = topics which are extracted already from previous responses

Parallel state fusion

A single user response can update multiple fields (e.g., “It happened yesterday evening near Wakad, the car is drivable, no police report”). Your engine must:

- merge all extracted/entered updates into the state in one turn, and
- ensure the next question reflects the merged state (no stale-field questions).

TASK 0 — Define Claimant State + Field Rules (Foundation)

Goal

Make a stable set of fields so all later tasks are consistent

What to build (checklist of deliverables, all not necessary)

1. **Claim_state_schema.json**
    - Required vs optional fields
    - Field types, enums, allowed nulls
    - “Unknown / not provided / not applicable” conventions

2. **Domain vocabulary lists (candidate-defined)**
    - incident types (collision/theft/fire/flood/vandalism/…)
    - damage areas & severities
    - document types (FIR/police report/photos/repair estimate/…)
    - coverage-related flags (policy_active, own_damage, third_party, add-ons, etc.)

3. **priority_policy.md**
    - Simple explicit rules, e.g. “Safety/legal workflow blockers first”, “Fraud-consistency checks early”, “Documentation requirements before settlement steps”, etc.

TASK 1 — Create ~1200 Insurance Intake Questions (Agentic Corpus Build)

Goal

Generate a large question corpus with metadata. (Not handwritten one-by-one.)

Required categories

Create ≥ 1200 questions total with minimum coverage per category/incident_type

- when/where, moving/parked, weather/visibility, road type
- collision_dynamics
- direction of impact, speed estimate buckets, lane/turning, signal compliance
- damage_assessment
- damage zones, severity, drivable/towed, airbags, glass, underbody
- third_party_details
- other vehicle, driver, contact details, insurer known/unknown, hit-and-run
- legal_reporting & documentation
- police report/FIR, case number, witness details, photos/video availability
- policy & eligibility gates
- policy status, owner/driver relationship, permitted driver, usage type, exclusions
- fraud/consistency checks
- timeline inconsistencies, repeated claims, improbable combinations
- repair & settlement preferences
- workshop choice, cashless vs reimbursement, towing, inspection scheduling
- theft/fire/flood/vandalism specific
- forced entry signs, keys available, water level exposure, fire source, etc.

For each generated question, attach metadata

Per-question required JSON schema fields

Each question must be a JSON object

```
{
  "id": "Q0345",
  "text": "Was the other vehicle's registration number captured
(photo/notes)?",
  "question_field":"third_party_details_registration",
  "priority": 2,
  "triggers": {
    "incident_type": ["collision"],
    "third_party_involved": true,
    "required_fields_present": ["loss_datetime", "loss_location"]
  },
  "targets": {
    "fill_fields": ["third_party_vehicle_id"]
  },
  }
}
```

Expected output files

- question_bank_raw.jsonl (≥ 1200 lines; 1 JSON per line)

TASK 2 — Validate Questions + Metadata

Goal

Catch wrong metadata, duplicates, nonsense triggers.

What to build

A validator pipeline that produces:

1. valid questions
2. rejected questions with reasons
3. duplicate removal

Validation must include

A) Schema validation

- required mandatory keys exist
- Data is correct (priority int, triggers, etc.)

B) Trigger/target logic validation

Examples of invalid:

- Question on followup asks about “collision” but it is defined in category/already_extracted_category field

```
{
  "id": "Q0345",
  "text": "Was the other vehicle's registration number captured
(photo/notes)?",
  "question_field":"third_party_details_registration",
  "priority": 2,
  "triggers": {
    "incident_type": ["collision"],
    "third_party_involved": true,
    "required_fields_present": ["loss_datetime", "loss_location"]
  },
  "targets": {
    "fill_fields": ["third_party_vehicle_id"]
  },
  }
}
```

If *third_party_details_registration* is already defined in the input category, it should not ask the question above

Claim_state after 3 followup_questions:

```
{
  "category": "collision",
  "loss_datetime": "Feb 3rd, 2026",
  "loss_location": {"city": "Pune", "road_type": "urban"},
  "vehicle": {"make": "Maruti", "model": "Baleno", "year": 2021},
  "drivable": true,
  "third_party_involved": true,

"already_extracted_categories":"collision","third_party_details_registratio
n",
  "answered_question_ids": ["Q0008"]
}
```

C) Duplicate / near-duplicate removal

- exact text duplicates removed

- near-duplicates removed using embeddings OR string similarity threshold

D) Coverage check

Ensure minimum coverage:
- each category has at least N questions (e.g., ≥100)
- red_flags must exist in some of the questions

Expected outputs

1. Question_bank_validated.jsonl

Target File

- Validated bank is clean and measurable
- You can explain why any rejected question was rejected

TASK 3 — Question Retrieval Engine (Pick the best next question)

Goal
Given a claim state, fetch the single best next question from the validated corpus.

What to build

retriever.py implementing:

- get_next_question(claim_state, question_bank_validated) -> retrieval_output
- retrieval_output_schema.json (defines returned JSON format)

Retrieval must work in two stages (clear + debuggable)
Stage 1: Hard filter (must satisfy)
Only keep questions where:
1. triggers match claim state
2. question not in answered_question_ids

Stage 2: Rank (choose best one)
Return exactly one question and include score_breakdown in output.

Expected outputs

- retriever.py

- retrieval_output_schema.json (defines the returned JSON format)

Target Build

- For 20 test states, engine picks logical questions
- The questions asked are logical and the rules and code are well defined and not repetitive, with unique questions being triggered for every scenario (not the same questions being asked for every combination of categories)
- It does not ask questions whose target fields are already known/answered
  Example: If user already mentions “I am a smoker and have fever and cough”
  Do NOT ask a question again like “Do you smoke” or “Do you have fever or cough”

TASK 4 — Manual Input Demo Loop

Goal

Prove retrieval works

What to build

A small CLI script:

1. User manually enters patient state fields
2. System returns next question
3. User manually updates state with new info
4. Repeat until termination

Example cycle:

Turn 0 — Initial manual claim_state input

```
{"input": "My car had an accident and got hit from the side from a bike"}
```

This should update the details in the fields (for reference only)

```
{
  "category": "collision",
  "loss_datetime": NA,
  "drivable": NA,
  "third_party_involved": true,
  "already_extracted_categories": ["collision"],
  "answered_question_ids": []
}
```

System returns next question

```
{
  "id": "Q0345",
  "text": "Was the other vehicle's registration number captured
```

```
(photo/notes)?",
  "question_field": "third_party_details_registration",
  "priority": 2,
  "triggers": {
    "incident_type": ["collision"],
    "third_party_involved": true,
    "required_fields_present": ["loss_datetime", "loss_location"]
  },
  "targets": {
    "fill_fields": ["third_party_details_registration"]
  }
}
```

User answers

```
{
  "input": "Yeah it was MH12AB1234, the accident happened today on Mar 2,
2026",
}
```

State update after Turn 0

```
{
  "category": "collision",
  "loss_datetime": "March 2nd, 2026",
  "loss_location": {"city": NA, "road_type": NA},
  "vehicle": {"make": NA, "model": NA, "year": NA},
  "drivable": NA,
  "third_party_involved": true,
  "third_party_vehicle_id": "MH12AB1234",
  "evidence_photos_available": NA,
  "already_extracted_categories": ["collision",
"third_party_details_registration"],
  "answered_question_ids": ["Q0345"]
}
```

—----------

Turn 1 — Next question should now avoid asking registration again, or the date of the accident since they are already mentioned

```
{
  "id": "Q0410",
  "text": "Was any police report filed, or was this handled privately
```

```
without reporting?",
  "question_field": "legal_reporting_status",
  "priority": 2,
  "triggers": {
    "incident_type": ["collision"],
    "required_fields_present": ["loss_datetime", "loss_location"]
  },
  "targets": {
    "fill_fields": ["police_report_filed"]
  }
}
```

User answers

```
{
  "input": "minor collision, exchanged details. We did not call police"
}
```

State after Turn 1

```
{
  "category": "collision",
  "loss_datetime": "Mar 2nd, 2026",
  "loss_location": {"city": "Pune", "road_type": "urban"},
  "vehicle": {"make": NA, "model": NA, "year": NA},
  "drivable": NA,
  "third_party_involved": true,
  "third_party_vehicle_id": "MH12AB1234",
  "evidence_photos_available": NA,
  "police_report_filed": false,
  "already_extracted_categories": ["collision",
"third_party_details_registration", "legal_reporting_status"],
  "answered_question_ids": ["Q0345", "Q0410"]
}
```

Turn 2 — Next question (damage scope). Make sure already extracted categories are not asked, and that the questions are dynamic based on the previous inputs.

```
{
  "id": "Q0522",
  "text": "Which parts are damaged on the side? (Select all: left door/
front bumper / rear bumper / headlamp / other)",
  "question_field": "damage_area_multi_select",
```

```
  "priority": 3,
  "triggers": {
    "incident_type": ["collision"],
    "required_fields_present": ["loss_datetime", "loss_location"]
  },
  "targets": {
    "fill_fields": ["damage_areas"]
  }
}
```

Expected outputs

1. Demo_manual_loop.py
2. sample_runs/ logs:
   Turn_01.json
   Turn_02.json

Target Built

- Demonstrates multiple turn flows
- Shows question selection + why
- Shows state updates + answered question tracking
- Dynamic questioning based on the inputs. If different responses are provided in a separate session, different and response related questions must be asked, so the question fetching algorithm needs to be effective to fetch based on inputs.
  Example, if user mentions somewhere during Turn 3 that there was a fire related incident that damaged the car, fetch questions related to a fire incident from the corpus based on priority and previous responses

TASK 5— Termination policy (When to stop asking questions)

Goal

System must decide: “enough info collected”.

What to build

should_terminate(state, history) -> {bool, reason}

Target Build

The interview stops reliably and doesn’t ask infinite low-value questions.

Task 6– Generation of different test cases

To test the built system with multiple various cases, make sure the system is robust to work in different situations and type of inputs

Example:

Case 1: Identical repeat input → should not ask the same question again or questions related to already answered fields

Turn 0:

```
{
  "case_id": "T6_001",
  "claim_state": {
    "category": "collision",
    "loss_datetime": "Feb 3rd, 2026",
    "loss_location": {"city": "Pune", "road_type": "urban"},
    "third_party_involved": true,
    "already_extracted_categories": ["collision"],
    "answered_question_ids": []
  },
  "expected_next_question": {
    "question_field": "third_party_details_registration"
  }
}
```

Turn 1: Same as Turn 0

```
{
  "case_id": "T6_001",
  "claim_state": {
    "category": "collision",
    "loss_datetime": "Feb 3rd, 2026",
    "loss_location": {"city": "Pune", "road_type": "urban"},
    "third_party_involved": true,
    "already_extracted_categories": ["collision"],
    "answered_question_ids": []
  },
  "expected_next_question": {
    "question_field": "third_party_details_registration"
  }
}
```

Case 2: Correction / negation → remove third-party involvement

This tests “field removal due to correction”.

Turn 0:

```
{
  "case_id": "T6_002",
  "claim_state": {
    "category": "collision",
    "loss_datetime": "Feb 4th, 2026",
    "loss_location": {"city": "Delhi", "road_type": "urban"},
    "third_party_involved": true,
    "already_extracted_categories": ["collision"],
    "answered_question_ids": []
  },
  "expected_next_question": {
    "question_field": "third_party_details_registration"
  }
}
```

User correction: “Actually it was a wall, no other vehicle.”
Claim State after correction

```
{
  "case_id": "T6_002_after",
  "claim_state": {
    "category": "collision_wall",
    "loss_datetime": "Feb 4th, 2026",
    "loss_location": {"city": "Delhi", "road_type": "urban"},
    "third_party_involved": false,
    "third_party_vehicle_id": null,
    "already_extracted_categories": [""],
    "answered_question_ids": []
  },
  "expected_non_questions": [
    "third_party_details_registration"
  ],
  "expected_next_question": {
    "question_field": "damage_area_multi_select"
  }
}
```

We are interested in your approach and reasoning of the methodology you choose. The approach must be quick, lightweight and accurate with a document stating your approach for the above tasks and logic behind your route taken.